Introduction to Artificial Intelligence

Lecture 3 · Logic programming

Relations · resolution · constraints

Today

01	**Programming with relations**	Facts, rules and queries; one definition, many uses
02	**The language**	Terms, atoms, clauses and what they mean
03	**How Prolog computes**	Unification, resolution and search trees
04	**Search in Prolog**	Graphs, water jugs, iterative deepening in one line
05	**Constraint logic programming**	N-queens, SEND + MORE = MONEY, Sudoku

01

Programming with relations

Say what, not how

An imperative program says *how* to compute an answer, step by step. A logic program states *what* is true about a problem, and the system searches for the answers.

> **“Algorithm = Logic + Control”**
>
> Robert Kowalski, 1979

- **Logic**: facts and rules about the problem. You write this part.
- **Control**: how to use them to answer a question. Prolog provides it: depth-first search with backtracking, as in lecture 1.
- Prolog, *programming in logic*, was created in Marseille in 1972.

Facts and rules

```
% Facts: parent(P, C) means that P is a parent of C.
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).
parent(pat, jim).

% Rules
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).

ancestor(X, Y) :- parent(X, Y).
ancestor(X, Z) :- parent(X, Y), ancestor(Y, Z).
```

- A **fact** states that a relation holds.
- A **rule** defines a relation from others: **:-** reads *if*, and the comma reads *and*.
- Two clauses for **ancestor**: a parent, or a parent of an ancestor. Recursion, with no loops and no data structures.

Asking questions

```
?- parent(tom, X).
X = bob ;
X = liz.

?- parent(X, ann).
X = bob.
```

```
?- grandparent(tom, W).
W = ann ;
W = pat ;
false.

?- ancestor(X, jim).
X = pat ;
X = tom ;
X = bob ;
false.
```

Variables start with an uppercase letter. Typing **;** asks for another answer, and **false** means there are no more.

Relations, not functions

Query	The question it asks
parent(tom, bob)	Is Tom a parent of Bob?
parent(tom, X)	Who are Tom’s children?
parent(X, ann)	Who are Ann’s parents?
parent(X, Y)	Which pairs are a parent and a child?

One definition answers all four questions. In Java, each would need its own method, or its own data structure.

One definition, many uses

```
append([], Ys, Ys).
append([X|Xs], Ys, [X|Zs]) :- append(Xs, Ys, Zs).
```

Appending an empty list to Ys gives Ys; appending [X|Xs] to Ys gives X followed by the result of appending Xs to Ys.

```
?- append([1,2], [3], L).
L = [1, 2, 3].

?- append(X, Y, [1,2]).
X = [],
Y = [1, 2] ;
X = [1],
Y = [2] ;
X = [1, 2],
Y = [] ;
false.

?- append(_, [Last], [a,b,c]).
Last = c ;
false.
```

The same two clauses concatenate lists, split a list in every possible way, and find its last element.

02

The language of logic programs

Terms

Kind	Examples	Written as
Atom (a constant)	tom, ann, [], 'New York'	a word in lowercase, or anything in single quotes
Number	42, 3.14	
Variable	X, Ys, _Rest, _	an uppercase letter or _ first; _ alone is anonymous
Compound term	parent(tom, bob), point(1, 2)	a functor and its arguments: point/2 has arity 2

Lists are compound terms too: **[1, 2, 3]** is **[1|[2|[3|[]]]]**, a head followed by a tail, and the pattern **[H|T]** splits a list into the two.

Atoms and clauses

> An **atomic formula** is a predicate applied to terms, such as **parent(tom, X)**; in logic programming it is just called an *atom*. Careful: in Prolog, *atom* also means a constant such as **tom**.

Clause	Example	Logical reading
Fact	parent(tom, bob).	parent(tom, bob) is true
Rule	grandparent(X, Z) :- parent(X, Y), parent(Y, Z).	for all X, Y, Z: parent(X, Y) ∧ parent(Y, Z) → grandparent(X, Z)
Query	?- grandparent(tom, W).	is there a W with grandparent(tom, W)?

Horn clauses

$$H \leftarrow B_1 \wedge B_2 \wedge \cdots \wedge B_n$$

- Prolog clauses are *Horn clauses*: one conclusion H and a conjunction of conditions. A fact has n = 0.
- The variables of a clause are universally quantified; those of a query, existentially: Prolog looks for values that make the query true.
- **The meaning of a program** is the set of ground atoms (atoms without variables) that follow logically from it: its *least Herbrand model*. Prolog's answers are logical consequences of the program.

Two readings of a clause

Declarative

grandparent(X, Z) holds if, for some Y, **parent(X, Y)** and **parent(Y, Z)** hold.

Procedural

To prove **grandparent(X, Z)**, first prove **parent(X, Y)**, then prove **parent(Y, Z)** with the Y just found.

Ideally, only the declarative reading matters. In practice, the procedural reading decides efficiency, and even termination.

03

How Prolog computes

Unification

```
?- f(X, b) = f(a, Y).
X = a,
Y = b.

?- point(X, X) = point(1, 2).
false.

?- [H|T] = [1, 2, 3].
H = 1,
T = [2, 3].

?- X = f(X).
X = f(X).
```

- Two terms **unify** if some substitution of their variables makes them identical. Prolog finds the *most general* one.
- Same functor and number of arguments, then unify the arguments pair by pair; a variable unifies with any term, and becomes bound to it.
- **X = f(X)** succeeds, building an infinite term: Prolog skips the *occurs check* for speed. **unify_with_occurs_check/2** performs it.

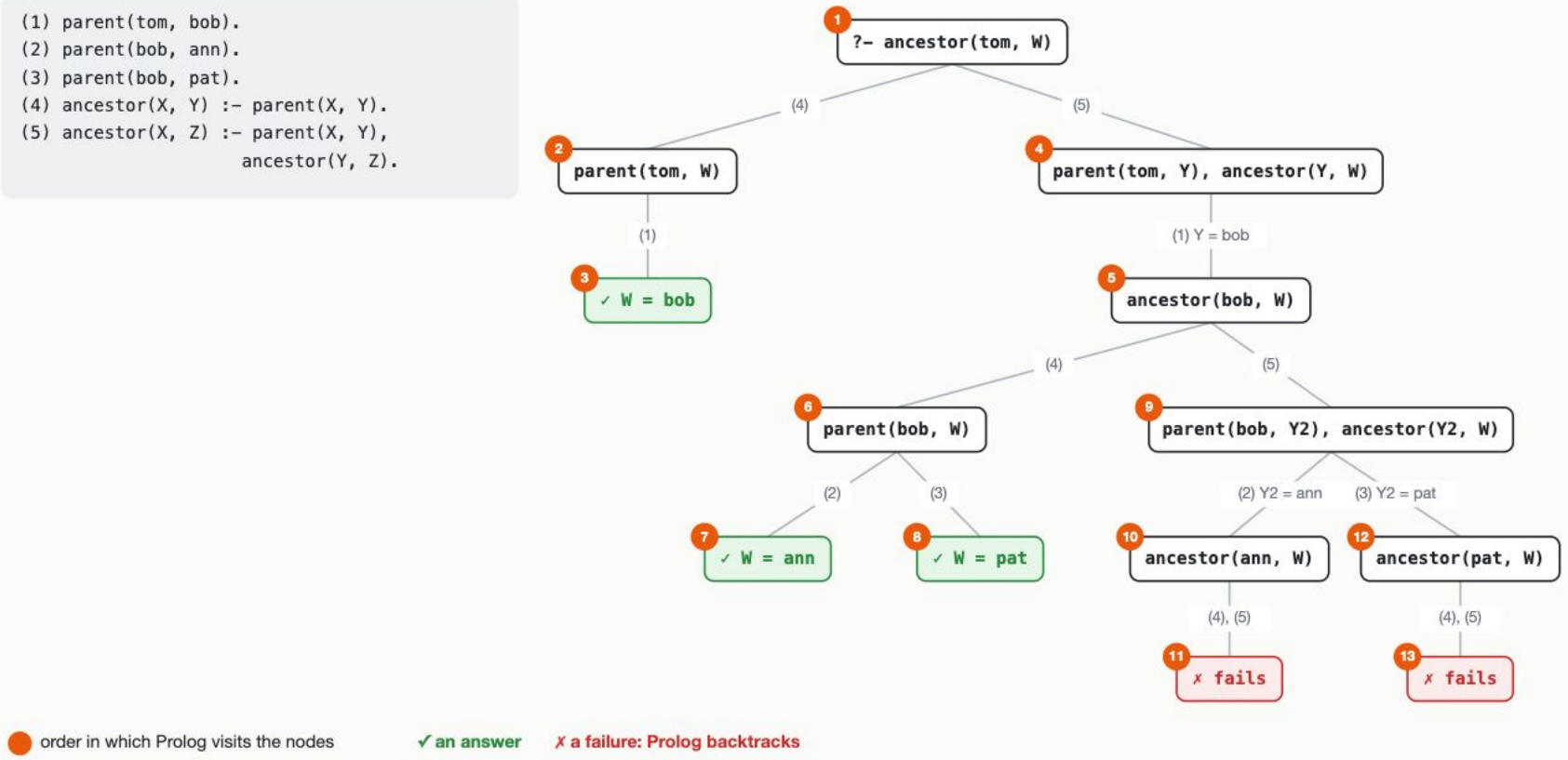
Resolution

```
function SOLVE(goals):
    if goals is empty: return success
    G ← the first goal
    for each clause H :- B1, ..., Bn, from top to bottom:
        rename the variables of the clause
        if G and H unify, with most general unifier θ:
            if SOLVE(θ(B1, ..., Bn, the other goals)): return success
    return failure
```

- Leftmost goal first, and clauses from top to bottom.
- Every clause that unifies is a branch. When a branch fails, Prolog *backtracks* to the last choice and tries the next clause.
- After an answer, typing `;` resumes the search at the last choice.

The search tree

```
(1) parent(tom, bob).
(2) parent(bob, ann).
(3) parent(bob, pat).
(4) ancestor(X, Y) :- parent(X, Y).
(5) ancestor(X, Z) :- parent(X, Y),
                      ancestor(Y, Z).
```



Depth-first, left to right: the answers are bob, ann and pat, in that order.

Sound and complete, in theory

> **Theorem.** SLD resolution is *sound*: every answer is a logical consequence of the program. It is also *complete*: every answer that follows from the program is in some branch of the SLD tree.

But Prolog visits the tree depth-first. If an infinite branch comes before an answer, Prolog never reaches the answer: Prolog itself is sound, but not complete.

This is the weakness of depth-first search from lecture 1.

Order matters

```
ancestor(X, Z) :- ancestor(X, Y), parent(Y, Z).
ancestor(X, Y) :- parent(X, Y).
```

```
?- ancestor(tom, W).
ERROR: Stack limit (1.0Gb) exceeded
ERROR:   Probable infinite recursion (cycle):
ERROR:     [6,340,043] user:ancestor(tom, _12693890)
ERROR:     [6,340,042] user:ancestor(tom, _12693910)
```

The same logical meaning as before, only in another order: the recursive call comes first, so Prolog calls **ancestor(tom, Y)** again and again, and never looks at a **parent** fact.

04

Search in Prolog

Paths in a graph

```
edge(a, b).
edge(b, c).
edge(c, a).
edge(c, d).

path(X, Y) :- edge(X, Y).
path(X, Y) :- edge(X, Z), path(Z, Y).
```

```
?- path(a, d).
true ;
true ;
true ;
...

?- path(a, e).
ERROR: Stack limit (1.0Gb) exceeded
```

The graph has a cycle, a → b → c → a. **path(a, d)** has infinitely many proofs, one per lap around the cycle, and **path(a, e)** never fails: Prolog keeps going round.

Remembering where we have been

```
path(X, Y, Path) :- walk(X, Y, [X], Reversed), reverse(Reversed, Path).

walk(X, X, Visited, Visited).
walk(X, Y, Visited, Path) :-
    edge(X, Z), \+ member(Z, Visited),
    walk(Z, Y, [Z|Visited], Path).
```

```
?- path(a, d, P).
P = [a, b, c, d] ;
false.

?- path(a, e, P).
false.
```

- **\+ G** succeeds if G cannot be proved: *negation as failure*.
- Here: never move to a node that is already on the path.

The water jugs in Prolog

```
% A state is jugs(X, Y): X liters in the 5-liter jug, Y in the 3-liter jug.
move(jugs(_, Y), fill1,  jugs(5, Y)).
move(jugs(X, _), fill2,  jugs(X, 3)).
move(jugs(_, Y), empty1, jugs(0, Y)).
move(jugs(X, _), empty2, jugs(X, 0)).
move(jugs(X, Y), pour12, jugs(X1, Y1)) :- T is min(X, 3 - Y), X1 is X - T, Y1 is Y + T.
move(jugs(X, Y), pour21, jugs(X1, Y1)) :- T is min(Y, 5 - X), X1 is X + T, Y1 is Y - T.

goal(jugs(4, _)).
goal(jugs(_, 4)).
```

Each move is one clause: the state before, the name of the action, the state after. **is** evaluates an arithmetic expression.

Search for free

```
solve(S, []) :- goal(S).
solve(S, [A|As]) :- move(S, A, S1), solve(S1, As).

solve_id(S, As) :- length(As, _), solve(S, As).
```

```
?- solve(jugs(0, 0), As).
ERROR: Stack limit (1.0Gb) exceeded

?- solve_id(jugs(0, 0), As).
As = [fill1, pour12, empty2, pour12, fill1, pour12] ;
As = [fill1, fill1, pour12, empty2, pour12, fill1, pour12] ;
```

length(As, _) creates lists of length 0, 1, 2, ..., and for each one **solve** searches depth-first, but only that deep: *iterative deepening in one line*. The first answer is a shortest solution.

N-queens by generate and test

```
queens(N, Qs) :- numlist(1, N, Rows), permutation(Rows, Qs), safe(Qs).

safe([]).
safe([Q|Qs]) :- no_attack(Q, Qs, 1), safe(Qs).

no_attack(_, [], _).
no_attack(Q, [Q1|Qs], D) :-
    Q =\= Q1 + D, Q =\= Q1 - D,
    D1 is D + 1, no_attack(Q, Qs, D1).
```

permutation/2 generates every arrangement of the rows, and **safe/1** checks the diagonals afterwards: blind generate and test. For 8 queens, up to 8! = 40,320 candidates.

05

Constraint logic programming

Constrain, then search

Generate and test builds complete candidates and checks them afterwards. **Constraint logic programming** states the constraints first: every choice then shrinks the possible values of the other variables, before they are tried.

CLP(FD), for integers	Meaning
X in 1..8, Xs ins 1..8	the domain of a variable, or of a list of variables
#=, #\=, #<, #>	arithmetic constraints, in any direction
all_different(Xs)	the variables take different values
label(Xs)	search for values, propagating after each choice

How constraints work

```
?- X in 1..5, Y in 1..5, X #> Y + 2.
X in 4..5,
Y#=<X+ -3,
Y in 1..2.
```

- **Propagation**: each constraint removes values that its variables can no longer take. Before any search, X > Y + 2 leaves X in 4..5 and Y in 1..2.
- **Search**: **label/1** tries values, one variable at a time, propagating after each choice, and backtracks on failure.
- Backtracking search plus inference: *constraint satisfaction*, a topic of a later week.

N-queens with constraints

```
:- use_module(library(clpfd)).

queens(N, Qs) :-
    length(Qs, N), Qs ins 1..N,
    safe(Qs),
    labeling([ff], Qs).

safe([]).
safe([Q|Qs]) :- no_attack(Q, Qs, 1), safe(Qs).

no_attack(_, [], _).
no_attack(Q, [Q1|Qs], D) :-
    Q #\= Q1, abs(Q - Q1) #\= D,
    D1 is D + 1, no_attack(Q, Qs, D1).
```

N	Generate and test	CLP(FD)
8	0.007 s	0.003 s
10	0.09 s	0.002 s
11	0.54 s	0.003 s
12	5.3 s	0.007 s
13	52 s	0.009 s
100	—	0.18 s

Time to the first solution, in SWI-Prolog. Generate and test grows about tenfold with every extra queen.

SEND + MORE = MONEY

```
send_more_money([S,E,N,D,M,O,R,Y]) :-
    Vars = [S,E,N,D,M,O,R,Y],
    Vars ins 0..9,
    all_different(Vars),
    S #\= 0, M #\= 0,
              1000*S + 100*E + 10*N + D
            + 1000*M + 100*O + 10*R + E
    #= 10000*M + 1000*O + 100*N + 10*E + Y,
    label(Vars).
```

```
?- send_more_money(L).
L = [9, 5, 6, 7, 1, 0, 8, 2] ;
false.
```

9567 + 1085 = 10652. There is no search code at all, and Prolog also proves that this is the only solution.

Sudoku in a dozen lines

```
sudoku(Rows) :-
    length(Rows, 9), maplist(same_length(Rows), Rows),
    append(Rows, Cells), Cells ins 1..9,
    maplist(all_distinct, Rows),
    transpose(Rows, Columns), maplist(all_distinct, Columns),
    Rows = [R1, R2, R3, R4, R5, R6, R7, R8, R9],
    boxes(R1, R2, R3), boxes(R4, R5, R6), boxes(R7, R8, R9),
    maplist(label, Rows).

boxes([], [], []).
boxes([A1,A2,A3|As], [B1,B2,B3|Bs], [C1,C2,C3|Cs]) :-
    all_distinct([A1,A2,A3,B1,B2,B3,C1,C2,C3]),
    boxes(As, Bs, Cs).
```

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

The black digits were given; Prolog found the orange ones in about 0.03 s. The program only states the rules: rows, columns and boxes hold different digits.

Relations run backwards

```
n_factorial(0, 1).
n_factorial(N, F) :- N #> 0, N1 #= N - 1, F #= N * F1, n_factorial(N1, F1).
```

```
?- n_factorial(5, F).
F = 120 ;
false.

?- n_factorial(N, 120).
N = 5 ;
false.
```

With constraints, arithmetic becomes a relation too: the same two clauses compute a factorial, and find the number whose factorial is 120.

Summary

- A logic program states facts and rules about relations; a query asks which values make it true.
- Terms, atoms and Horn clauses, with a declarative and a procedural reading.
- Prolog answers by resolution and unification, searching the SLD tree depth-first: sound, but not complete, so the order of clauses and goals matters.
- Search problems become short programs, and iterative deepening takes one line.
- Constraint logic programming states constraints before searching, and solves puzzles like N-queens and Sudoku with propagation plus search.

In the lab

1. Family relations: sibling, ancestor, and questions in every direction.
2. Lists with **append/3**: the last element, prefixes, membership, one line each.
3. The water jugs: write the pouring moves, and find shortest solutions with iterative deepening.
4. N-queens with constraints, timed against generate and test.
5. Puzzles: run SEND + MORE = MONEY and Sudoku, then write BASE + BALL = GAMES.

Next: local search

- When only the final configuration matters, not the path: N-queens, timetables, designs.
- Keep a single current state, and improve it step by step: hill climbing, simulated annealing, genetic algorithms.

Reading: Bratko, *Prolog Programming for Artificial Intelligence*, chapters 1–3; Learn Prolog Now!, chapters 1–6.